Pruebas de Restricciones y Automatización

Verificación de reglas de negocio, acciones en cascada y disparadores en la base de datos

1. Restricciones de Tuplas

1.1. Caso TuplasOK

Aquí probamos que la regla funciona cuando hacemos bien. Cambié el estado de un arma a 'Dañada', pero me aseguré de que no tuviera ningún soldado asignado. Como esto cumple perfectamente con la regla que creamos, la base de datos me dejó guardar el cambio sin problemas.

```
PS C:\Users\juant> & "C:\Program Files\PostgreSQL\18\bin\psql.exe" -U postgres -d proyecto_militar
Password for user postgres:

psql (18.6)
WARNING: Console code page (850) differs from Windows code page (1252)
         8-bit characters might not work correctly. See psql reference
         page "Notes for Windows users" for details.
Type "help" for help.

proyecto_militar=# UPDATE armamento SET estado_arma = 'Dañada', id_soldado = NULL WHERE id_arma = 2;
UPDATE 1
```

1.2. Caso TuplasNoOK

Acá queríamos ver qué pasa si intentamos romper la regla. Intenté asignarle un arma que está 'Dañada' al soldado 15. Como era de esperarse, la base de datos se dio cuenta de la trampa, bloqueó la acción y me soltó un error. Así nos aseguramos de nunca entregarle equipo averiado a nadie.

```
proyecto_militar=# UPDATE armamento SET estado_arma = 'Dañada', id_soldado = 15 WHERE id_arma = 8;
ERROR:  el nuevo registro para la relación «armamento» viola la restricción «check» «chk_arma_danada_sin_soldado»
DETAIL:  La fila que falla contiene (8, WPN-37355946, Dañada, 10, 15).
proyecto_militar=#
```

2. Acciones Referenciales

2.1. Borrado en Cascada (AccionesOK)

En esta prueba verificamos si el borrado en cascada está haciendo su trabajo. Eliminé por completo la misión 445 y, como podemos ver, la base de datos se encargó de borrar automáticamente todas las asignaciones de soldados que estaban vinculadas a esa misión. Así evitamos que queden datos "fantasma"

```
proyecto_militar=# DELETE FROM mision WHERE id_mision = 445;
DELETE 1
proyecto_militar=# SELECT * FROM asignacion_mision WHERE id_mision = 445;
 id_asignacion | rol_asignado | id_mision | id_soldado
---------------+--------------+-----------+------------
(0 rows)
```

3. Disparadores (Triggers)

3.1. Validaciones DisparadoresOK

Aquí probamos que el sistema automático (el Trigger) nos deja trabajar cuando todo está en orden. Intenté asignarle una nueva misión al soldado 15. Como la base de datos revisó su historial médico y vio que está sano, lo dejó pasar y guardó la asignación con éxito.

```
proyecto_militar=# INSERT INTO asignacion_mision (id_asignacion, rol_asignado, id_mision, id_soldado) VALUES (9001, 'Asalto', 1, 15);
INSERT 0 1
```

3.2. Bloqueo DisparadoresNoOK

Intenté mandar a una misión al soldado 1, pero él está de 'Baja Médica'. El sistema inmediatamente frenó la asignación y me mostró el mensaje de error personalizado que configuramos, demostrando que la automatización protege la salud de los soldados.

```
proyecto_militar=# INSERT INTO asignacion_mision (id_asignacion, rol_asignado, id_mision, id_soldado) VALUES (9002, 'Asalto', 1, 1);
ERROR:  AutomatizaciÃ³n: El soldado 1 estÃ¡ de Baja MÃ©dica y no puede ir a misiones
CONTEXT:  función PL/pgSQL verificar_salud_soldado() en la línea 11 en RAISE
```

4. Comandos de Mantenimiento

Se incluyen además las sentencias para deshabilitar o eliminar limpiamente los disparadores y sus funciones asociadas de la base de datos en caso de requerir mantenimiento futuro.